

BELGIUM CAMPUS iTVERSITY

SOFTWARE ENGINEERING 381

ASSIGNMENT 3

Research for Quality, Security and Production Readiness Decisions

Project: CivicConnect

Team: Virtual Group K

Academic year: 2026

Submission date: 25 September

Team members

Cameron Purdon — 601963

Michael Ellis — 600332

Bianca Grobler — 600537

Non-participation declaration:

All listed team members made a meaningful contribution to the assignment. No non-participation is reported.

Table of Contents

Table of Contents	1
Question 1- Quality Strategy & Risk-Based Verification	2
1.1 How QA, QC, V&V and testing relate	3
1.2 Risk-based verification	3
1.3 Complementary Verification Evidence	3
1.4 Using Verification Methods Together	4
1.5 Automation and quality gates	5
1.5.1 Gates must define a progression decision	5
1.6 Risk to verification evidence map	6
1.7 Critical Question: Why is "all automated tests passed" insufficient?	6
Question 2 Security Engineering and Threat Based Controls.....	7
2.1 Purpose of Threat Modelling.....	7
2.2 STRIDE Threat Modelling.....	7
2.2.1 How STRIDE Supports Engineering Decisions	7
2.2.2 Limitations of STRIDE	8
2.3 Chain 1: Authentication	8
2.4 Chain 2: Secrets / Configuration	8
2.5 Additional concerns Authorisation and access control.....	9
2.6 Input and data handling	9
2.7 Threat-to-Control Traceability Table	10
2.8 Secure Design, Secure Coding and Security Checks	10
2.9 Limitations of Automated Security Tools and AI	11
2.10 Critical question, what remains unproven if a scanner reports no high-severity findings?	11
Question 3 Production Readiness Deployment and Operational Evidence.....	11
3.1 Environment strategy and parity	11
3.2 Secrets and production configuration	12
3.3 Release and deployment	12
3.3.1 Repeatability and release controls.....	12
3.4 Comparing deployment approaches.....	13

3.5 Failure and recovery	13
3.6 Operational quality: reliability, scalability, cost, supportability	14
3.7 Production Readiness, Observability and Operational Evidence.....	14
3.7.1 Observability	14
3.7.2 Logs	14
3.7.3 Metrics.....	15
3.7.4 Monitoring	15
3.7.5 Alerts	15
3.8 Production-Readiness Evidence Matrix	16
3.9 Critical Question, Why Functional Software May Be Unready for Production?	17
Question 4 – Research-to-M3 Engineering Map	18
AI Research and Verification Record.....	21
Reference List	22

Question 1- Quality Strategy & Risk-Based Verification

1.1 How QA, QC, V&V and testing relate

These terms get used interchangeably a lot, but they sit at different levels. Quality Assurance is the process-level umbrella. It is about preventing defects before they happen, through things like standards, reviews and process discipline. Quality Control sits underneath QA and is product-oriented: it is the set of activities that check whether what was actually built matches what was required, and it is a failure-detection system rather than a prevention one, since QA is process oriented and comes before QC, which measures quality by verifying that what was implemented matches requirements.

Verification and Validation are the two halves of QC. Verification asks "did we build the product right?", checking conformance to the spec, while validation asks "did we build the right product?", checking whether it actually meets the user's need. Verification checks whether a system meets its specifications, while validation checks whether the right product was built to meet high-level requirements. In practice this maps onto testing activities fairly cleanly: verification tends to rely on static techniques like code review, while validation is confirmed through dynamic activities like user acceptance testing, though most real test types, system testing especially, blend both. The important point for a quality strategy is that none of QA, QC, V&V or testing alone guarantees quality. Verification and validation by themselves do not guarantee software quality; that also depends on planning, traceability and configuration management as part of the wider engineering process. That is the argument for why quality has to be planned in from the start rather than bolted on as a testing phase at the end.

1.2 Risk-based verification

Once you accept that you cannot test everything to the same depth (time and budget do not allow it), the question becomes: what gets more attention? Risk-based testing answers this by scoring risk as a function of two things: the likelihood of a problem occurring, which mostly comes from technical factors like languages and infrastructure, and the impact of that problem, which comes from business factors like financial loss and number of users affected. ISTQB formalises this as risk level, defined as the importance of a risk based on its likelihood and impact, used to decide how much test attention it deserves, rather than leaving prioritisation to gut feeling or the order features happened to be built in.

The practical effect is that a payment or authentication feature used by every user gets deep, layered verification, while a rarely-used admin cosmetic feature gets a light pass. This is the mechanism that connects "quality expectations and consequences of failure" (as the brief phrases it) to actual verification effort, and it is the concept your Risk-to-Verification Evidence Map should be built around. For each risk, ask what its likelihood and impact are, and let that decide the depth and type of evidence collected.

1.3 Complementary Verification Evidence

Software quality should not depend on a single testing method. Different verification techniques identify different types of defects and therefore provide different forms of evidence about the quality of a system. A quality strategy should combine several complementary verification methods so that important risks are identified at different stages of development. The assignment requires consideration of what each method is effective at identifying, as well as what it cannot prove on its own.

Table 1: Comparison of Complementary Verification Types

Verification type	What it can identify effectively	What it may miss or cannot prove
Unit testing	Identifies defects within individual functions, methods or components. It is useful for detecting incorrect calculations, conditions, boundary cases and unexpected behaviour within isolated pieces of code.	It does not demonstrate that different components communicate correctly, that APIs work correctly, or that the complete user workflow functions as expected.
Integration/API testing	Identifies problems between components, services, databases and APIs. It can expose incorrect data formats, interface problems and failures to comply with API contracts.	It may not identify defects within an individual component's internal logic. It also does not necessarily demonstrate that the complete system satisfies user requirements.
System/end-to-end testing	Tests complete workflows across the integrated system. It can identify problems that only occur when several components operate together and provides evidence that system-level requirements are being met.	It can be slower and more difficult to diagnose when a failure occurs. It also cannot prove that every possible user scenario or defect has been tested.
Static analysis	Examines source code without executing the application. It can identify certain coding defects, potentially unsafe coding practices, unreachable code and some security weaknesses.	It cannot demonstrate that the application behaves correctly during execution. It may also miss integration, usability, performance and environment-specific problems.
Performance/load testing	Identifies performance problems under expected or increased workloads, including slow response times, insufficient throughput and excessive resource consumption.	It does not prove that the application's functionality or security is correct. Results can also depend on whether the test environment and workload accurately represent production conditions.
Regression testing	Determines whether existing functionality has been affected by new changes to the system.	It can only identify problems covered by the regression tests. If an important scenario is not included in the regression suite, the defect may remain undetected.

ISTQB identifies different levels and types of testing because different testing activities provide evidence about different aspects of software behaviour. Testing can provide confidence in a system, but it cannot prove that a system contains no defects (ISTQB, 2024).

1.4 Using Verification Methods Together

These verification methods should be viewed as complementary rather than interchangeable. For example, a unit test could demonstrate that an authentication function correctly rejects an incorrect password. An integration test could then verify that the authentication service communicates correctly with the user database. An end-to-end test could subsequently verify that a user can complete the entire login process through the application.

Static analysis provides another layer of evidence by identifying potential problems in the source code without executing the application. Performance testing then considers a different risk by examining how the application behaves under increased demand.

This supports a risk-based verification strategy. Verification effort should be influenced by the likelihood and impact of failure. Functions that have greater business, technical, security or operational consequences should receive more extensive verification than lower-risk functionality. This approach is consistent with the assignment requirement that verification should be based on product risk rather than attempting to test everything equally.

1.5 Automation and quality gates

Automation makes selected checks repeatable, but the checks must assert the behaviour that matters. A useful pipeline starts from a controlled revision, restores declared dependencies in a clean environment, runs relevant checks and retains the results. Reports should identify the revision, test data, environment, tool versions, failures and skipped tests so that reviewers can assess what actually ran. The Week 4 notes distinguish this evidence from simply executing tests (SEN381 Teaching Team, 2026a).

Fast unit, static and focused integration checks suit frequent pull requests. Broader workflow, security and representative load checks can be applied to a release candidate where their additional cost is justified. This reduces feedback delay, but postponing a high-impact check until release can make correction expensive. The split should therefore follow change impact: an ownership update deserves concurrent database verification even if it changes only a few lines.

1.5.1 Gates must define a progression decision

A quality gate needs an explicit criterion, required evidence, a decision owner and a consequence for failure. Illustrative merge blockers include an unreproducible build, an unauthorised record disclosure or a partial request/history save. A minor formatting finding might generate tracked work instead. Severity labels alone are insufficient: a finding affecting a core access boundary may matter more than its generic tool rating. Release review additionally needs evidence about deployment, recovery and user acceptance; permission to merge is a narrower decision.

GitHub supports required reviews and status checks, with strict checks requiring an up-to-date branch. Its documented acceptance of successful, skipped or neutral check statuses also shows why a mergeable pull request does not establish that every intended test executed (GitHub, n.d.-a). A candidate safeguard is to inspect expected test counts and required job execution, and fail the evidence check when a mandatory suite is missing. These are capabilities and research options, not claims about the current repository settings.

Blocking every warning creates noise and encourages bypassing controls; blocking too little permits serious regressions. Flaky checks need investigation, an owner and a time-bound repair. If temporarily isolated, their

missing assurance must be recorded and replaced by suitable evidence. Repeatedly rerunning a failure until it passes hides uncertainty. Exceptions should record the affected risk, justification, reviewer and expiry. This follows the wider need to define security-check criteria and review evidence throughout development (Souppaya, Scarfone and Dodson, 2022).

1.6 Risk to verification evidence map

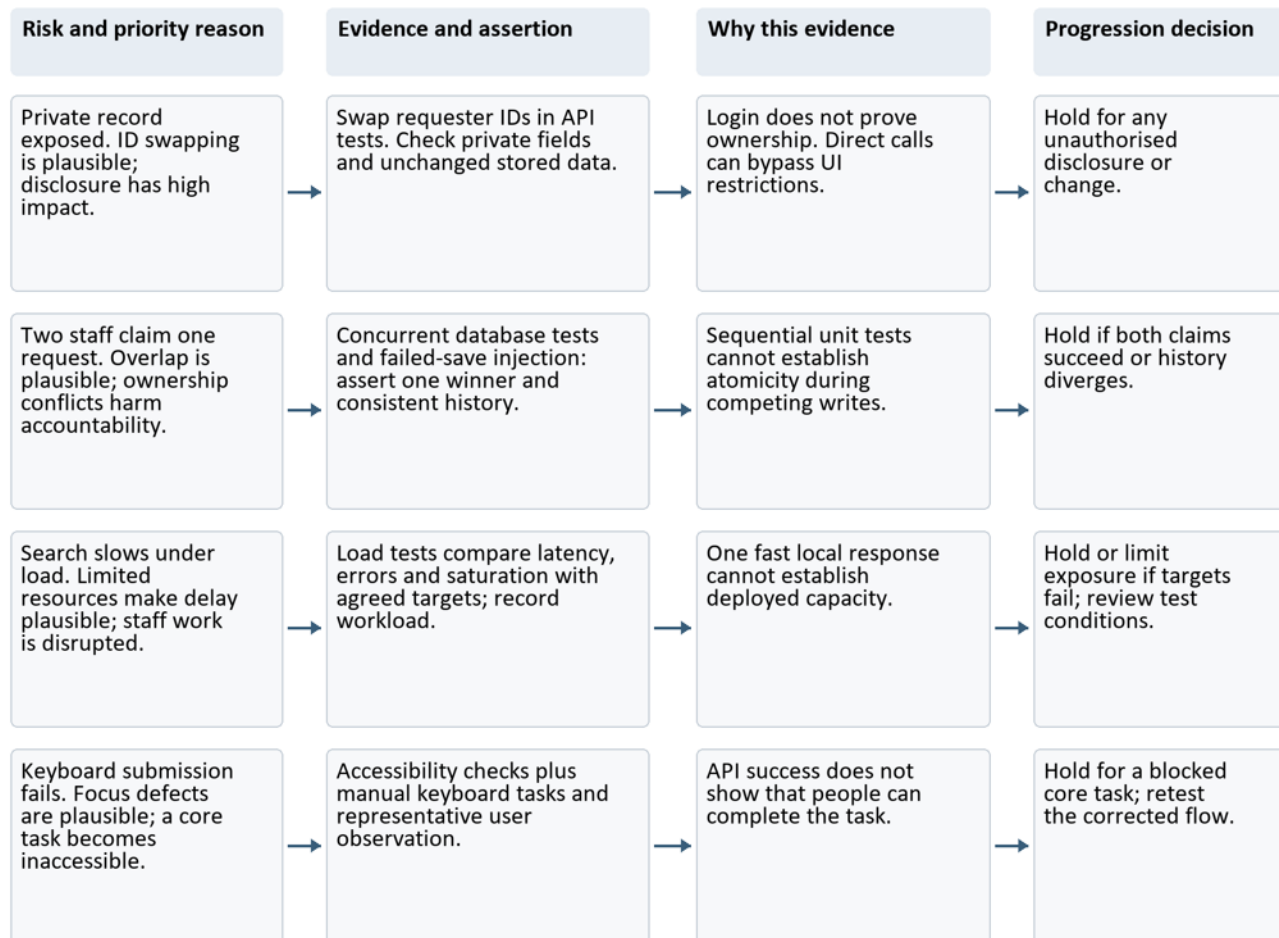


Figure 1. Risk-to-Verification Evidence Map. Original synthesis using ISTQB (2024, sections 1.3 and 5.2) and the course quality-gate guidance (SEN381 Teaching Team, 2026a).

1.7 Critical Question: Why is "all automated tests passed" insufficient?

A passing suite supports only the assertions actually executed on that revision, with those inputs and environmental conditions. It can share the implementation's mistaken assumptions, omit an important role or check only that a response exists. Even high code coverage does not establish meaningful assertions. ISTQB (2024, section 1.3) explains that testing cannot establish the absence of defects and that conformance to specified requirements can still fail to meet users' needs.

For example, successful request submission tests could coexist with private-note disclosure, conflicting ownership under concurrency or an unusable keyboard flow. The evidence in Figure 1 answers different questions. Release judgement must combine relevant automated results with independent review, user validation and operational evidence, then state the remaining risk. A green pipeline is therefore a qualified input to that judgement

Question 2 Security Engineering and Threat Based Controls

2.1 Purpose of Threat Modelling

Threat modelling is a structured approach used to identify potential security threats during software development. Rather than relying only on security testing after the system has been developed, threat modelling considers possible attacks and misuse scenarios during the design and development process.

This allows potential threats to be translated into security requirements, engineering controls and verification activities. Microsoft describes threat modelling as a methodical examination of a system's design or architecture to identify potential security problems (Hernan et al., 2006).

The assignment requires a recognised threat-modelling approach to be discussed in terms of how it supports engineering decisions, rather than simply providing a list of security threats.

2.2 STRIDE Threat Modelling

One recognised threat-modelling approach is STRIDE. STRIDE was developed by Microsoft and provides six categories that can be used to systematically consider possible threats to a system (Microsoft, 2022).

STRIDE category	Meaning	Example
S – Spoofing	Pretending to be another user or system.	An attacker obtains another user's login details and accesses the system using their identity.
T – Tampering	Unauthorised modification of information.	An attacker modifies data being sent through an API.
R – Repudiation	A user denies having performed an action.	A user claims that they did not perform a transaction, but the system has insufficient audit information to determine who performed it.
I – Information Disclosure	Unauthorised access to information.	Personal or confidential information is exposed through an API response.
D – Denial of Service	Preventing legitimate users from accessing the system.	An attacker sends excessive requests to consume system resources.
E – Elevation of Privilege	Obtaining permissions beyond those authorised.	A normal user exploits an access-control weakness to obtain administrative privileges.

STRIDE provides a structured way of asking security-related questions about system components, processes and data flows (Microsoft, 2022).

2.2.1 How STRIDE Supports Engineering Decisions

The main value of STRIDE is that an identified threat can be linked to a specific security requirement and engineering control.

For example:

Threat: Elevation of privilege

Security requirement: Users must only access resources for which they have permission.

Engineering control: Role-based access control and server-side authorisation checks.

Verification evidence: Tests demonstrating that users cannot access restricted resources.

This creates a traceable relationship between the threat, requirement, control and verification evidence, which is the approach required by the assignment.

2.2.2 Limitations of STRIDE

Although STRIDE provides a useful structure for identifying common security threats, it does not guarantee that every security weakness will be identified. Threat modelling still depends on the quality of the system information and the reasoning performed by the people conducting the analysis.

For example, a threat that does not clearly fit into one of the STRIDE categories could be overlooked. In addition, STRIDE does not replace secure coding, dependency checking, security testing or penetration testing.

Therefore, STRIDE should be regarded as a structured method for supporting security analysis, rather than a complete replacement for other security activities.

2.3 Chain 1: Authentication

A weak authentication implementation is one of the most consistently exploited weaknesses in web systems. Broken authentication is one of the most severe threats to web applications and APIs according to OWASP, and it typically shows up as things like failing to rotate session IDs after login or failing to properly invalidate session IDs at logout or after inactivity. The requirement this generates is straightforward: only legitimate, verified users should be able to establish and keep a session. The engineering control most commonly recommended is layered: multi-factor authentication is recommended because it prevents automated attacks like credential stuffing, since an attacker typically cannot complete an additional factor automatically, combined with proper session invalidation and hashed (not plaintext) credential storage. Verification evidence would include authentication-specific test cases (session timeout, logout invalidation, brute-force lockout) plus a dependency/config check confirming MFA and hashing libraries are actually wired in, not just present in the codebase.

2.4 Chain 2: Secrets / Configuration

The threat here is a secret, an API key, database password or token, ending up somewhere it can be read by unauthorised people, most commonly by hard-coded credentials being embedded directly in source code or configuration files. This is explicitly addressed as a common cause of account and system compromise. Once committed, the risk does not go away even if the line is later deleted, because the secret can remain accessible through version control history even after being removed from the current version of the file. The control objective

is to keep sensitive configuration completely separate from source code. The typical engineering control is to store and manage secrets using dedicated secret-management solutions such as secure vaults or cloud-native secret stores, and retrieve them at runtime through environment variables or managed identities rather than embedding them directly. Verification evidence includes automated secret-scanning in the CI pipeline (checking for committed keys) and a config review confirming no environment file with real secrets is tracked in source control.

2.5 Additional concerns Authorisation and access control

Threat → objective. An authenticated requester changes a record identifier to obtain another requester's details, or a staff member invokes an action outside their permitted category. The objective is to permit only authorised operations on authorised records and fields. Authentication establishes identity; it does not establish permission for every object. OWASP recommends least privilege, denial by default and permission checks on every request (OWASP Foundation, n.d.-a).

Candidate control. Server-side enforcement could combine roles with ownership, category and note-visibility rules. Role-only checks are simple but cannot distinguish two requesters holding the same role. Attribute or relationship rules express finer restrictions, although they require consistent policy definitions and more test combinations. A shared enforcement mechanism can reduce omissions; it must still receive the relevant record and action context. Hiding a button or using difficult-to-guess identifiers cannot replace authorisation (OWASP Foundation, n.d.-a).

Verification and residual risk. A candidate test matrix would vary identity, role, record ownership, category, operation and field visibility. Direct API tests should include permitted and denied cases, swapped identifiers, unauthenticated calls and private-note checks in both detail and list responses. Denial must leave stored data unchanged. Policy review should confirm that the expected results reflect stakeholder rules. Otherwise, tests can faithfully enforce an incorrect permission model. These examples connect to FR-003, FR-006, FR-008, FR-011, FR-016 and NFR-001 without choosing the M3 implementation.

2.6 Input and data handling

Threat → objective. A request description containing executable markup could run when staff view it; manipulated search input could alter a dynamically constructed query. A caller might also submit an unsupported category or force an invalid status change. The objective is to keep untrusted input as data and accept only values and changes consistent with the domain rules.

Candidate controls address different boundaries. Server-side validation can check type, length, allowed category and business meaning; client-side checks improve feedback but can be bypassed. Validation alone is not a primary defence against injection (OWASP Foundation, n.d.-c). Parameterised queries separate SQL instructions from values; dynamically chosen sort fields require a controlled mapping because identifiers are not ordinary value parameters (OWASP Foundation, n.d.-f). Context-appropriate output encoding and safe rendering prevent text being interpreted as markup or script. Rich HTML, if genuinely required, needs suitable sanitisation; treating descriptions as plain text is simpler but offers less formatting (OWASP Foundation, n.d.-b).

Verification and residual risk. Candidate evidence includes boundary-value and malformed-input tests, direct API attempts to bypass browser validation, and database-backed tests of search parameters. Browser tests should store representative malicious text and confirm it remains inert wherever displayed. Review must also inspect

query construction and unsafe rendering paths: a small payload set cannot prove that every context is protected. Rejected submissions should not partially persist, and user errors should not disclose internals. These examples relate to FR-001, FR-002, FR-007, FR-010, NFR-003 and NFR-008. M3 must establish actual limits and rendering contexts rather than importing arbitrary restrictions from a generic checklist.

2.7 Threat-to-Control Traceability Table

Threat / misuse	Security requirement	Engineering control	Verification evidence	Residual risk / limitation
Session not invalidated at logout/timeout, allowing session hijack	Only actively authenticated users can hold a valid session	Session expiry plus explicit invalidation on logout; MFA on login	Automated session-management test cases; auth-specific test suite	Does not prove MFA is enforced for every entry point (e.g. API vs UI)
Credential stuffing / brute-force login attempts	System resists automated mass login attempts	Rate limiting, account lockout and MFA	Load/abuse test simulating repeated login attempts	Lockout thresholds can themselves be abused for denial-of-service against a user
Hard-coded secrets committed to source control	Secrets never appear in application code or repo history	Secrets manager / vault with environment variable injection at runtime	Automated secret-scanning in CI; manual config review before release	Scanning only catches known patterns; a disguised or newly-formatted secret can slip through
Sensitive config values differing across environments causing accidental exposure	Environment-specific configuration is controlled and auditable	Centralised configuration management, separated per environment	Configuration diff/audit as part of release checklist	Does not prove the production values themselves are still correctly protected once deployed
Requester swaps an ID or staff exceeds category permission	Protect records, actions and private notes from unauthorised access	Server-side role plus ownership/category checks; deny by default	Positive and negative API matrix; inspect response fields and unchanged data	Incorrect policy or an omitted route can leave access gaps
Untrusted descriptions or search values alter code/query behaviour	Treat input as data and enforce domain-valid changes	Server validation, parameterised queries and context-safe rendering	Boundary and bypass tests; database integration and stored-input browser tests; code review	Payload samples miss contexts; valid format does not imply authorised action

2.8 Secure Design, Secure Coding and Security Checks

None of these three replaces the others. Code security is not one tool but a set of complementary checks that each look at a different facet of the code at different points in the pipeline, and no single one is sufficient. OWASP itself notes static analysis alone finds only a fraction of an application's flaws. Design principles establish things like least privilege and fail-secure defaults before a line of code is written; secure coding practices are the human discipline of actually implementing that design correctly (input validation, parameterised queries); and automated checks (SAST, dependency scanning, secret scanning) exist to catch what slips through both of the above. Tellingly, tooling cannot compensate for a team that keeps introducing the same class of flaw. A team writing injection-prone code will generate findings faster than any scanner can clear, which is the argument for treating these three as reinforcing layers, not substitutes.

2.9 Limitations of Automated Security Tools and AI

Coverage gap: SAST analyses source code without running it, looking for patterns that match known vulnerability classes, meaning it can only flag what matches a known pattern. Logic flaws, business-rule bypasses and novel vulnerability classes it has not been trained on go undetected.

False confidence in a clean result: A scan or AI recommendation reporting no findings does not mean the system is secure. It means nothing matching the tool's known checks was found. It says nothing about human-layer issues like weak default configuration decisions, business-logic authorisation bypasses, or context the tool has no visibility into.

2.10 Critical question, what remains unproven if a scanner reports no high-severity findings?

A clean scan proves the absence of the specific known vulnerability patterns the tool checks for. It does not prove the absence of business-logic flaws, authorisation bypasses that do not match a known signature, insecure design decisions, or how the system behaves under a genuinely creative attacker. It is evidence of one layer, not a certificate of overall security.

Question 3 Production Readiness Deployment and Operational Evidence

3.1 Environment strategy and parity

Development supports rapid changes and debugging. CI/test provides repeatable checks against controlled test data. Staging deploys a candidate for representative integration and release checks; production serves actual users under operational controls. These are distinct purposes, not necessarily four permanent servers (SEN381 Teaching Team, 2026b).

The Twelve-Factor guidance recommends reducing differences between development and production, including differences in backing services (The Twelve-Factor App, n.d.-b). A lightweight local database or permissive local configuration may hide behaviour that differs in the deployed database, permissions or network. Versioned setup scripts, declared runtime/dependency versions and a recorded configuration schema make these differences

inspectable. Containers may package runtime dependencies consistently, but cannot by themselves reproduce database state, certificates, service permissions or resource limits.

A permanent staging environment gives the team a stable place to rehearse deployment and check configuration drift, but costs resources and can itself become outdated. An environment created for each candidate provides cleaner isolation and can reduce idle cost, but takes time to provision and may omit persistent operational behaviour. For a small pilot, a shared staging instance may be proportionate if access, resets and concurrent changes are controlled. M3 must judge this against the actual host and team constraints.

Parity should preserve relevant behaviour while maintaining data and credential separation. Tests should use synthetic data and separate resources; copying production secrets to make environments “identical” creates avoidable exposure. Useful evidence includes a version/configuration inventory, justified differences and a deployment rehearsal with the target database engine and representative network restrictions. Any smaller staging capacity limits what its load results can support.

3.2 Secrets and production configuration

The Twelve-Factor approach separates deploy-specific configuration from code and proposes environment variables (The Twelve-Factor App, n.d.-a). OWASP adds a security qualification: environment variables can leak through logs or dumps, particularly in container environments, so protected mounted files or runtime retrieval from a secret store may be preferable (OWASP Foundation, n.d.-e). Separating a value from Git is only one part of protecting it.

A platform-managed secret store can simplify deployment integration, while a dedicated manager may offer stronger lifecycle controls at the cost of additional configuration and availability dependencies. M3 must compare access restrictions, auditability, rotation support and operational effort. Credentials should be scoped to their environment and purpose, with revocation and replacement available after exposure. A committed secret must be revoked or rotated; deleting the latest file does not invalidate copies (OWASP Foundation, n.d.-e).

Candidate release evidence would identify required configuration keys without exposing values, demonstrate refusal to start with missing critical settings, and confirm production debug behaviour, HTTPS and database permissions. A controlled rotation exercise should show the replacement works and the old credential no longer does. Repository scanning, deployed configuration review and log/error inspection provide complementary evidence. None justifies printing secrets into the evidence record. This extends the security objective in Q2 into operational verification and relates to NFR-006 and NFR-008.

3.3 Release and deployment

A successful build produces a software artefact. Deployment must place that artefact and compatible configuration into a working target environment; release exposes the resulting behaviour to users. Build success cannot establish correct routing, database access, migrations, HTTPS or a usable request workflow. Google’s release-engineering guidance emphasises reproducible, automated processes so changes can be repeated and understood (Beyer et al., 2016).

3.3.1 Repeatability and release controls

A candidate delivery process would identify a reviewed revision, build a versioned artefact, test it, deploy it to staging and promote that same artefact with controlled environment-specific configuration. Rebuilding separately for production introduces a further opportunity for dependency or build drift. A release record should connect the revision and artefact identifier to check results, configuration version, migration prerequisites, approval and deployment outcome. Database compatibility must be assessed before promotion, feeding into the team's recovery analysis.

Continuous delivery can retain an explicit human production-release decision. Automation removes repetitive deployment steps but cannot decide whether residual risk is acceptable. GitHub environments illustrate one implementation option: they can restrict deployment branches, require approval and withhold environment secrets until applicable protection rules pass. Availability varies with repository visibility and plan, so M3 must check eligibility before depending on those controls (GitHub, n.d.-b). This documents a capability, not a platform selection.

3.4 Comparing deployment approaches

Candidate approach	Potential benefit	Limitation affecting the decision
Replace the running version	Simple operation and lower simultaneous resource demand.	May interrupt service and expose all users to a defect. Needs a rehearsed recovery path.
Blue green deployment	Prepare a second environment and switch traffic after checks; previous application version can remain available.	Duplicates resources and requires routing/configuration discipline. Switching application versions does not undo database writes.
Canary deployment	Expose a small share of users or traffic first and compare candidate health with a baseline.	Requires representative traffic, routing and meaningful signals. Low pilot traffic can make the comparison inconclusive.

The canary trade-off follows Google's analysis of limiting release exposure while evaluating production signals (Beyer et al., 2018). The other rows express engineering alternatives to evaluate against available capacity, acceptable interruption and data compatibility; no approach guarantees a safe release.

After deployment, a candidate smoke check should exercise the actual user route: authenticate with a controlled account, submit a synthetic request, read its status and confirm protected data remains isolated. A live process or HTTP success at the root page is insufficient. M3 must define the checks, observation window, stop conditions and responsible person. Failed checks should stop further exposure and trigger the agreed recovery assessment; success still requires continued observation through Michael's proposed operational evidence.

3.5 Failure and recovery

The core problem with rollback is that application code and database state do not fail the same way. Rolling back code is simple; application rollback typically means replacing deployed artefacts with the previous working version. But if the broken deployment ran migrations that altered the database schema or data, rolling back the code does not undo those migrations. This is the single most important thing a team has to plan for before calling itself production-ready: a rollback plan that only covers application code is incomplete.

There are a few recognised strategies, each with trade-offs. Backup-and-restore is straightforward and works even for extensive changes, but restoring can cause downtime and lose any data created after the backup was taken.

Roll-forward with compensating scripts is faster than restoring from backup and allows partial reversion, but writing scripts that correctly handle every edge case is complex. A widely recommended pattern for avoiding the problem altogether is designing schema changes to be backward-compatible in the first place. This approach is sometimes called expand-and-contract or N-1 compatibility, where new and old code can both run against the schema during a transition window, removing the need for a risky database rollback at all. What should happen when a release fails, at minimum: an automated or well-rehearsed rollback path for the application, a clear decision rule for whether a database rollback or roll-forward fix is used, and a recent, tested backup as the last resort.

3.6 Operational quality: reliability, scalability, cost, supportability

Using ISO/IEC 25010 as a shared vocabulary, reliability measures the degree to which software performs its specified functions consistently under defined conditions over time, without interruptions and failures, and includes fault tolerance and recoverability as sub-characteristics. This directly connects to the rollback discussion above; recoverability is not a separate concern, it is part of what reliability actually means.

Scalability and performance are grouped under the same standard's performance efficiency characteristic, and in practice get expressed as latency budgets, load testing, caching and batching strategies for shipping systems that stay fast under load. A system that works fine in testing with five users but degrades badly at production scale has not actually demonstrated production readiness. It has demonstrated functional correctness only.

Cost and supportability are less standardised in the ISO model but matter just as much for a production-readiness decision. A system might be reliable and scalable but prohibitively expensive to run at the traffic level expected, or so poorly documented and monitored that supporting it in production requires knowledge that only exists in one developer's head. Both are practical constraints that sit alongside the more measurable quality attributes, and a production-readiness decision has to weigh them together rather than treating reliability and scalability as the whole story.

3.7 Production Readiness, Observability and Operational Evidence

Production readiness involves more than successfully developing and deploying an application. A production-ready system should be capable of being deployed, operated, monitored and recovered safely.

3.7.1 Observability

Observability refers to the ability to understand what is happening inside a running system by examining information produced by the system. Four important elements of operational observability are logs, metrics, monitoring and alerts.

3.7.2 Logs

Logs provide records of events that occur within an application or its supporting infrastructure. They can provide information about errors, requests, authentication events, system activities and other important events.

OWASP recommends application logging because logs can provide useful evidence for both security and operational investigations (OWASP Foundation, n.d.-d).

For example, an authentication failure could be recorded with information such as:

- date and time;
- account identifier;
- type of event;
- affected service; and
- relevant error information.

Logs are particularly useful when engineers need to determine what happened and why it happened.

3.7.3 Metrics

Metrics are numerical measurements collected over a period of time. Examples include:

- response time;
- number of requests;
- error rate;
- CPU usage;
- memory usage;
- database connections; and
- number of active users.

Google's Site Reliability Engineering guidance identifies latency, traffic, errors and saturation as important categories of system metrics for monitoring services (Beyer et al., 2016).

Metrics are particularly useful for identifying trends. For example, increasing response times over several days could indicate that a system is approaching a capacity limit.

3.7.4 Monitoring

Monitoring involves collecting, processing and displaying information about a system so that engineers can determine whether it is operating as expected.

Monitoring can combine information from logs and metrics to provide an overall view of system behaviour. Dashboards can, for example, display application errors, response times and resource utilisation.

Monitoring therefore provides more than simply collecting information; it allows engineers to determine whether the system is operating within acceptable conditions.

3.7.5 Alerts

Alerts notify the relevant people when a condition requires investigation or action.

For example:

Metric: API error rate

Threshold: Error rate exceeds the acceptable level

Alert: Engineering team receives a notification

Action: Team investigates the affected service.

Alerts should therefore be linked to conditions that require action. Excessive alerts that do not require investigation can create alert fatigue, where important warnings may be overlooked.

Google's SRE guidance emphasises that monitoring should provide information that is useful for detecting and responding to problems rather than simply generating large quantities of data (Beyer et al., 2016).

This is particularly relevant to the assignment because it states that collecting operational data without actionable interpretation is insufficient.

3.8 Production-Readiness Evidence Matrix

The assignment requires a minimum of six production-readiness concerns. For each concern, the matrix must identify the risk if the concern is ignored, evidence required before release, and evidence that should continue to be monitored after release.

Production-readiness concern	Risk if ignored	Evidence required before release	Evidence to monitor after release
Environment parity	Differences between development, testing and production environments can result in unexpected behaviour or deployment failures.	Successful testing in an environment representative of production and documented configuration differences.	Environment-specific errors, failed deployments and configuration-related incidents.
Secrets and configuration	Sensitive credentials may be exposed or incorrect configuration may prevent the application from operating correctly.	Evidence that secrets are separated from source code and correctly configured for the target environment.	Configuration failures, secret-access events and security alerts.
Deployment and release	A successful software build does not guarantee that the application will deploy or operate correctly.	A repeatable deployment process and successful deployment validation.	Deployment failures, application health and release-related errors.
Performance	The system may function correctly but become too slow when subjected to realistic workloads.	Load/performance test results against defined performance requirements.	Response times, throughput, error rates and resource utilisation.

Failure and recovery	System failures could result in extended downtime or loss of data.	Tested backup, rollback and recovery procedures.	Backup status, recovery events, failed deployments and recovery times.
Application errors	Failures may occur without the development or support team being aware of them.	Error-handling tests and evidence that significant errors are appropriately recorded.	Application error rates, exception logs and relevant alerts.
Security and audit logging	Security incidents may be difficult to investigate if there is insufficient evidence of system activity.	Verification that important security events are recorded appropriately.	Authentication failures, access-control events and suspicious activity.
Capacity and scalability	Increasing usage may exhaust system resources and cause degraded performance or service interruptions.	Load/stress testing and an assessment of expected capacity requirements.	Traffic levels, resource saturation and capacity trends.

The matrix demonstrates that production readiness requires both pre-release evidence and ongoing operational evidence. A system should not be considered ready simply because it has passed functional tests.

3.9 Critical Question, Why Functional Software May Be Unready for Production?

Why can software that passes functional tests and runs successfully on a developer's machine still be unready for production?

Functional testing provides evidence that the tested functionality behaves correctly under the conditions covered by the tests. However, a developer's environment may differ significantly from the production environment.

A system may therefore pass functional tests while still having:

- incorrect production configuration;
- improperly managed secrets;
- insufficient performance under realistic workloads;
- inadequate monitoring or alerting;
- an untested rollback procedure;
- database compatibility problems;
- insufficient backup and recovery capability; or
- environment-specific failures.

Production readiness therefore requires broader evidence than functional correctness. The system needs to be deployable, observable, recoverable, supportable and capable of operating reliably under expected production conditions.

This aligns with the assignment's distinction between software that is functional and software that is genuinely production-ready.

Question 4 – Research-to-M3 Engineering Map

This table converts the strongest findings from Questions 1 to 3 into a bridge for later Milestone 3 decision-making. It does not select a CivicConnect tool, test suite, security control or deployment platform; the final column states what Milestone 3 must still decide once actual project evidence is available.

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
Risk-based testing directs verification depth using likelihood and impact rather than treating all features equally.	Wasted verification effort on low-risk areas, insufficient effort on high-risk ones.	Risk scoring (likelihood x impact) used to prioritise which features get deeper test coverage.	Risk scoring is still a judgement call and can be under- or overestimated without real incident data.	Which CivicConnect features are actually high risk, and who decides the scoring.
Verification and validation each answer a different question, so passing one does not confirm the other.	Software may satisfy documented requirements while still failing to meet users' actual needs.	Combine static/verification techniques (code review) with dynamic/validation techniques (acceptance testing).	Blending both takes more time and does not remove the need for judgement on what "done" means.	How much acceptance testing CivicConnect needs before a feature counts as verified.
Broken authentication controls (session handling, lack of MFA) are a leading cause of account compromise.	Authentication / session security risk.	MFA plus proper session invalidation on logout and timeout.	Adds friction for users and does not prove every entry point (API vs UI) is equally protected.	Whether CivicConnect requires MFA for all users or only certain roles.

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
Hard-coded secrets in source control remain exposed even after being removed, because they persist in commit history.	Secrets and configuration exposure risk.	Secrets manager/vault with environment variable injection at runtime.	Secret scanners only catch known patterns; a disguised secret can still slip through.	Which secrets management approach fits CivicConnect's hosting environment and budget.
Application rollback and database rollback are not the same thing; migrations are not automatically undone by reverting code.	Release failure and recovery risk.	Backward-compatible schema changes (expand and contract) plus a tested backup as last resort.	Backup restore can cause downtime and lose data created after the backup point.	What CivicConnect's rollback plan looks like once the database schema is finalised.
ISO/IEC 25010 treats reliability (fault tolerance, recoverability) and performance/scalability as separate but related quality attributes.	Production readiness beyond “it runs on my machine.”	Load testing plus defined recovery/availability targets before release.	Testing at small scale does not guarantee behaviour at real production load.	What reliability and scalability targets are realistic for CivicConnect's expected user base.
Different verification methods identify different types of defects, so no single verification method provides complete quality evidence.	Important defects may remain undetected when verification relies on only one form of evidence.	Consider complementary evidence such as unit testing, integration/API testing, system testing and static analysis.	Each method adds execution and maintenance cost, and its conclusions remain limited by the scenarios, data and environments covered.	Decide which verification methods are appropriate for CivicConnect based on its actual risks and requirements.
A green automated test pipeline does not prove that the software is	Release confidence and quality assurance	Consider combining automated testing with static analysis, performance testing,	Additional verification requires time and resources, while some quality	Decide what quality evidence should be required before a CivicConnect

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
completely reliable or production ready.		security checks and other relevant evidence.	characteristics cannot be fully demonstrated through automated tests.	release can pass the M3 quality gate.
STRIDE provides a structured way of identifying security threats and linking them to requirements, controls and verification evidence.	Security and threat-based engineering	Consider threat modelling using STRIDE and linking identified threats to security requirements, controls and verification tests.	STRIDE provides a structured framework but cannot guarantee that every possible security weakness will be identified.	Decide which threats are relevant to CivicConnect and which security controls and verification evidence should be applied.
Automated security tools cannot prove that a system is completely secure.	Security assurance and residual risk	Consider combining automated security scanning with threat modelling, secure coding practices and security verification.	Automated tools can miss design-level or context-specific weaknesses and may produce false positives or false negatives.	Decide how automated security evidence should be combined with other security assurance activities in M3.
Production readiness requires more than successfully building and deploying the application; the system must also be observable and recoverable.	Deployment and operational readiness	Consider deployment evidence, rollback and recovery procedures, logs, metrics, monitoring and actionable alerts.	Additional operational controls require development and maintenance effort and may increase system complexity.	Decide what production-readiness and recovery evidence CivicConnect needs before release.
Logs, metrics, monitoring and alerts provide different types of operational evidence	Observability and	Consider defining relevant logs and metrics, monitoring dashboards and alerts	Collecting excessive information can create noise and	Decide which operational information is important for

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
and should support actionable decisions.	operational support	for significant failures or performance problems.	alert fatigue, while insufficient monitoring can leave important failures undetected.	CivicConnect and which conditions should trigger alerts.
Role checks alone do not establish record-level permission (OWASP Foundation, n.d.-a).	Unauthorised access between users with the same role.	Ownership/category/field rules with a positive and negative API permission matrix.	Fine-grained rules increase combinations and can encode an incorrect policy.	Which subject, record and action rules apply, where will they be enforced, and what test matrix is sufficient?
A build must be followed by target-environment deployment evidence (Beyer et al., 2016; SEN381 Teaching Team, 2026b).	A reproducible artefact may still fail when released.	Promote an identified artefact; rehearse configuration/migrations and run user-route smoke checks.	Representative staging costs effort; remaining environment differences limit confidence.	Which environment, promotion controls, smoke checks and stop conditions fit the actual CivicConnect implementation?

AI Research and Verification Record

Team Member	AI tool and use	Purpose	Output Used?	How independently verified	What was changed or rejected
Cameron Purdon (601963)	OpenAI Codex was used to help interpret the Assignment 3 requirements, structure assigned sections and drafts as well as create the visual creation of the Risk-to-Verification Evidence Map	To help create a rough idea of what needs to be done and help find relevant information that could be looked at	Partly, all content was reviewed and redone to match more closely with what was required.	All sources were checked and opened to make sure the claims were accurate and course notes and independent research was done to make sure everything used was accurate	Generic explanations, false claims, irrelevant or over wordy information
Bianca Grobler (600537)	Microsoft Copilot: source leads and research assistance	Starting point for research and drafting	Partly, after rewriting	Opened the sources independently and checked the claims against the original text	Used the research as a starting point and rewrote the content based on her own understanding
Bianca Grobler (600537)	ChatGPT: assistance with the layout of the Q4 table	Formatting the research findings into the required table structure	Yes	Checked each row against her completed research sections	Retained her own research but arranged it under the correct Q4 table columns
Michael Ellis (600332)	ChatPDF: assistance with reviewing and evaluating potential citations	To identify potentially relevant sources and information for further investigation	Partly. It was used only to identify possible citations; no AI-generated content was used as a source	Opened the original documents and personally verified the relevant information against the source text	Rejected the AI-generated wording and citation content. Only information verified in the original sources was used

Reference List

- Beyer, B., Jones, C., Petoff, J. and Murphy, N.R. (eds.) (2016) *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media. Available at: <https://sre.google/sre-book/table-of-contents/> (Accessed: 21 September 2026).
- Beyer, B., Murphy, N.R., Rensin, D.K., Kawahara, K. and Thorne, S. (eds.) (2018) *The Site Reliability Workbook: Practical Ways to Implement SRE*. Sebastopol, CA: O'Reilly Media. Available at: <https://sre.google/workbook/table-of-contents/> (Accessed: 28 September 2026).
- GitHub (n.d.-a) 'About protected branches', *GitHub Docs*. Available at: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches> (Accessed: 28 September 2026).
- GitHub (n.d.-b) 'Deployments and environments', *GitHub Docs*. Available at: <https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments> (Accessed: 28 September 2026).
- Hernan, S., Lambert, S., Ostwald, T. and Shostack, A. (2006) 'Uncover security design flaws using the STRIDE approach', *MSDN Magazine*, November. Available at: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2006/november/uncover-security-design-flaws-using-the-stride-approach> (Accessed: 21 September 2026).
- ISTQB (2024) *Certified Tester Foundation Level Syllabus*. Version 4.0.1, 15 September. Available at: https://istqb.org/?download_id=3345&sdm_process_download=1 (Accessed: 28 September 2026).
- ISTQB (n.d.) *ISTQB Glossary*. Available at: https://glossary.istqb.org/en_US/search (Accessed: 28 September 2026).
- Microsoft (2022) 'Threats: Microsoft Threat Modeling Tool', *Microsoft Learn*, 25 August. Available at: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats> (Accessed: 21 September 2026).
- Microsoft (n.d.) 'Secure by design', *Microsoft Security Development Lifecycle*. Available at: <https://www.microsoft.com/en-us/securityengineering/sdl/practices/secure-by-design> (Accessed: 28 September 2026).
- OWASP Foundation (n.d.-a) 'Authorization Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (Accessed: 28 September 2026).

OWASP Foundation (n.d.-b) 'Cross Site Scripting Prevention Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html (Accessed: 21 September 2026).

OWASP Foundation (n.d.-c) 'Input Validation Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html (Accessed: 28 September 2026).

OWASP Foundation (n.d.-d) 'Logging Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html (Accessed: 28 September 2026).

OWASP Foundation (n.d.-e) 'Secrets Management Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html (Accessed: 28 September 2026).

OWASP Foundation (n.d.-f) 'SQL Injection Prevention Cheat Sheet', *OWASP Cheat Sheet Series*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (Accessed: 28 September 2026).

SEN381 Teaching Team (2026a) *Collaborative Integration, Continuous Integration, Automated Verification and Quality Gates*. Week 4, Day 2, SO20–SO21. Belgium Campus iTversity. Supplied course material.

SEN381 Teaching Team (2026b) *Peer Review, Practical Continuous Delivery and Engineering Review*. Week 4, Day 3, SO22–SO24. Belgium Campus iTversity. Supplied course material.

Souppaya, M., Scarfone, K. and Dodson, D. (2022) *Secure Software Development Framework SSDF Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218. Gaithersburg, MD: National Institute of Standards and Technology. Available at: <https://doi.org/10.6028/NIST.SP.800-218> (Accessed: 28 September 2026).

The Twelve-Factor App (n.d.-a) 'III. Config'. Available at: <https://12factor.net/config> (Accessed: 28 September 2026).

The Twelve-Factor App (n.d.-b) 'X. Dev/prod parity'. Available at: <https://12factor.net/dev-prod-parity> (Accessed: 28 September 2026).